

Ratatosk — Semantic Discovery and Manifest Generation

Overview

Ratatosk is the discovery layer of the Mimir Labs architecture stack. It is designed to extract, normalize, and formalize the semantic meaning of enterprise data before large-scale system transformation occurs.

Most enterprise software initiatives fail not because of tooling limitations but because organizations cannot articulate the meaning of their own data structures. Definitions for fundamental business entities such as "customer", "inventory", or "available quantity" often differ across departments, systems, and reporting layers. Ratatosk addresses this problem by creating a structured, machine-readable description of enterprise meaning.

The output of Ratatosk is a **canonical semantic manifest** describing entities, attributes, relationships, and operational constraints across the enterprise data environment.

This manifest becomes the foundation for downstream systems in the Mimir Labs stack, including migration tooling, synchronization layers, and governance engines.

Role in the Mimir Labs Architecture

Ratatosk operates as the entry point into the broader architecture stack.

The stack is structured as follows:

1. Ratatosk — semantic discovery and manifest generation
2. Mímisbrunnr — canonical semantic reference model
3. Ragnarok — migration and transformation engine
4. Bifrost — cross-system synchronization layer
5. Jormungandr — governance and lifecycle enforcement
6. Yggdrasil ERP — operational execution environment

Ratatosk analyzes existing enterprise systems and produces a semantic manifest aligned to the canonical reference model provided by Mímisbrunnr.

Downstream systems use this manifest to guide data transformation, synchronization, and governance.

Problem: Semantic Drift

Enterprise environments accumulate multiple operational systems over time. ERP platforms, warehouse systems, CRM tools, analytics pipelines, and custom applications all maintain their own data definitions.

Although the structures may appear compatible, the meaning of key fields frequently diverges.

Common examples include:

- "Customer" representing a billing entity in finance but a purchasing contact in CRM
- "Available inventory" meaning different things depending on reservations, quality holds, or warehouse states
- "Revenue" being recognized differently between operational reporting and accounting systems

These semantic inconsistencies propagate through integrations and reporting layers, eventually producing operational conflict.

ERP migrations frequently expose these conflicts only after implementation has begun, when correction becomes expensive and disruptive.

Ratatosk moves this discovery process earlier by making semantic inconsistencies visible before migration or integration begins.

Discovery Workflow

Ratatosk is typically deployed through a structured discovery engagement.

The discovery process proceeds through several stages.

System Mapping

Existing systems are cataloged, including:

- ERP platforms n- warehouse management systems
- CRM systems
- manufacturing execution systems
- analytics environments

Schema structures and operational data models are captured.

Entity Extraction

Core business entities are identified across systems. Examples include customers, suppliers, orders, products, inventory locations, and financial accounts.

Each entity definition is examined for structural and semantic differences.

Semantic Alignment

Entities are mapped to canonical definitions derived from the Mímisbrunnr reference model.

Conflicts in meaning, lifecycle state, or operational constraints are documented.

Manifest Generation

The discovery process produces a machine-readable manifest describing the enterprise semantic model.

This artifact becomes the authoritative description of enterprise meaning.

Canonical Manifest

The primary output of Ratatosk is the **Ratatosk Manifest**, a JSON document describing the semantic structure of enterprise data.

The manifest includes:

- entity definitions
- attribute mappings
- cross-system relationships
- lifecycle states
- transformation rules
- validation constraints

Example structure:

```
{
  "entities": [
    {
      "name": "Customer",
      "canonical_model": "Mimisbrunnr.Customer",
      "sources": [
        "erp.customer_master",
        "crm.accounts"
      ],
      "attributes": {
        "id": "uuid",
        "name": "string",
        "status": "enum"
      }
    }
  ]
}
```

```
]
}
```

This manifest is designed to be consumed by automation systems and migration tools.

Integration with Mímisbrunnr

Mímisbrunnr provides the canonical semantic model used to normalize enterprise data structures.

Ratatosk aligns discovered enterprise entities with this reference model.

This alignment allows organizations to translate disparate schemas into a shared semantic representation without forcing immediate structural changes to existing systems.

The canonical model therefore acts as a semantic intermediary between systems with incompatible schemas.

Integration with Migration and Synchronization Layers

Once a Ratatosk manifest has been produced, additional components of the architecture stack can operate against it.

Ragnarok

Ragnarok uses the manifest to generate transformation pipelines for ERP migrations or system replacements.

Bifrost

Bifrost uses the manifest to maintain semantic synchronization between multiple operational systems.

Jormungandr

Jormungandr uses the manifest to enforce governance policies and maintain semantic consistency over time.

Operational Architecture

Ratatosk is implemented as a modular desktop application built using C++ and Qt. The tool interacts with enterprise systems through database connections, API integrations, and structured metadata extraction.

Key subsystems include:

- schema discovery engine
- semantic alignment engine
- quality analysis engine
- manifest generation engine

These components operate together to extract and formalize enterprise data meaning.

Deployment Model

Ratatosk can operate in several deployment configurations.

Typical modes include:

- consultant-operated discovery environment
- internal enterprise governance tool
- pre-migration analysis platform

Because Ratatosk operates primarily on metadata and schema information, it can perform meaningful analysis without requiring full operational access to production systems.

Strategic Role

Ratatosk represents the first step in establishing coherent enterprise data architecture.

By formalizing the meaning of enterprise entities before transformation occurs, organizations gain a stable semantic foundation upon which migrations, integrations, and operational systems can be built.

The Ratatosk manifest becomes a durable artifact describing enterprise meaning — one that can guide both current operations and future system evolution.

Ratatosk therefore functions not merely as a discovery tool, but as the mechanism through which organizations translate informal operational knowledge into a formal semantic architecture.